

Reflection Report: Software Testing Project

Şeymanur Şaşmaz

May 19, 2026

1 Introduction

The ShopEasy project provided a practical environment to apply modern software testing methodologies. We integrated specification-based testing, structural code analysis, code coverage metrics, Design by Contract, Property-Based Testing (PBT), and mocks. This report evaluates these applied strategies, analyzes the critical bugs discovered, and outlines our future testing roadmap.

2 Bug Discovery and Technique Analysis

During the development process, we observed that different testing approaches demonstrated asymmetrical success in catching bugs within the source code. Most notably, Property-Based Testing (PBT), implemented using the jqwik library, brought to light an extremely critical logical flaw hidden deep within the ShoppingCart class.

While attempting to verify the "Cart Commutativity" rule in the system, jqwik generated random product pools and, purely by chance, produced product objects that possessed the exact same unique identifier (ID) but different price tags. When these objects were added to the cart in varying sequences, we discovered that the final total amount (`total()`) of the cart fluctuated and mathematical inconsistencies occurred (e.g., a result that should have been 0.06 was calculated as 0.03). Upon examining the root cause of the error, it became clear that the cart mechanism based its calculation solely on the price of the first product entering the system for a specific ID, completely ignoring subsequent price updates.

This critical detail completely slipped under the radar of traditional Unit Tests. When a developer manually constructs a test scenario, they naturally tend to assign unique IDs to each distinct product. Similarly, Mock structures simulating external service scenarios failed to catch this vulnerability because they do not interact with the internal mathematical calculations of the cart. This situation proves PBT's absolute superiority in discovering extreme edge cases that exceed the boundaries of manual human foresight. Although this might seem like a rare occurrence, a malicious user who notices an outdated product price record left in the database could exploit this bug to manipulate the system and gain an unfair advantage.

3 Effectiveness and Efficiency

Property-Based Testing was highly effective for catching complex logic errors with minimal test code, saving hours of manual scenario writing.

Conversely, traditional unit tests supported by **JaCoCo** integration were highly efficient for broad coverage. Achieving a solid **86% Instruction Coverage** and **76% Branch Coverage** overall in the `ShoppingCart` class instantly highlighted untested areas. For instance, metric gaps in the `addItem` (72% branch) and `removeItem` (50% branch) methods were easily closed with simple assertions. Ultimately, Unit Tests act as a quick and reliable seatbelt, while PBT serves as an X-ray machine for deep architectural blind spots.

4 Testing Pyramid and Gap Analysis

Our current tests predominantly sit at the bottom of the testing pyramid: the Unit Test level. We successfully tested components like `OrderProcessor` in isolation using mock objects. However, we have notable gaps in the upper layers:

- **Integration Tests:** We lack tests verifying communication with real databases or external payment APIs.
- **End-to-End (E2E) Tests:** We are missing live scenarios simulating a real user's journey from interface clicks to completing a purchase.

5 Future Testing Roadmap

With additional development time, I would strictly prioritize establishing **Integration Tests**. Testing the `OrderProcessor` with isolated databases and realistic API simulators is crucial. Production errors typically occur at junction points where independent systems exchange data. Testing how the system handles network timeouts or database connection failures would elevate ShopEasy into a robust, production-ready software.

6 Conclusion

This project demonstrated that no single testing methodology is sufficient alone. Unit Tests and Code Coverage provide an agile foundation, Mocks effectively isolate components, and PBT catches hidden logic bugs. Combining these into a hybrid strategy—and eventually adding high-level integration tests—is essential for building software that is completely resilient against real-world anomalies.